Lösungen der Warm-up-Karten

Gleiche Nummerierung wie auf den Karten. Fachlich richtige Alternativen sind gleichwertig – entscheidend sind die genannten Kernpunkte.

W0-09 Die Waage des Alchemisten · Tracetabelle

b) d1: a / b ist eine Division zweier Ganzzahlen – das Ergebnis 3 wird erst danach in die Kommazahl 3.0 umgewandelt. d2: Der Cast wirkt nur auf a (Cast bindet stärker als /), also wird 17.0 / 5 als Kommazahl gerechnet: 3.4. d3: Die Klammer erzwingt zuerst die ganzzahlige Division (3), der Cast macht daraus 3.0. Merke: (int) schneidet die Nachkommastellen ab, es wird nicht gerundet (g = 3).

c) (int) (Math.random() * 6) + 1 – Math.random() liefert eine Kommazahl von 0 (einschließlich) bis 1 (ausschließlich). Mal 6 ergibt 0 bis unter 6, der Cast schneidet ab: 0, 1, …, 5. Plus 1 ergibt 1 bis 6. Allgemein für von bis bis: (int) (Math.random() * (bis − von + 1)) + von.

```
Variable | Wert
---------+-----
q        | 3
r        | 2
d1       | 3.0
d2       | 3.4
d3       | 3.0
g        | 3
ok       | true
```

W0-10 Die Sanduhr der Akademie · Lückencode

a) (1) 3600 · (2) % · (3) / · (4) rest % 60 (gleichwertig: sekunden % 60)

b) (sekunden + 59) / 60 – die ganzzahlige Division rundet ab; durch das Addieren von 59 wird jeder angebrochene Rest auf die nächste volle Sanduhr gehoben. 120 → 179 / 60 = 2 · 121 → 180 / 60 = 3 · 0 → 59 / 60 = 0.

Alternative mit Verzweigung: uhren ← sekunden / 60; falls sekunden % 60 ≠ 0, dann uhren ← uhren + 1.

```
public static String zeitText(int sekunden) {
    int h = sekunden / 3600;
    int rest = sekunden % 3600;
    int min = rest / 60;
    int s = rest % 60;
    return h + " h " + min + " min " + s + " s";
}
```

W0-11 Der Türsteher der Bibliothek · Fehler im Java-Code

Fehler 1: Reihenfolge der Bedingungen – jedes Kind unter 6 ist auch unter 18, landet also im ersten Zweig; der Zweig alter < 6 wird nie erreicht. preis(4, false) liefert 3 statt 0. Korrektur: zuerst alter < 6 prüfen.

Fehler 2: mitglied = true ist eine Zuweisung, kein Vergleich. Sie setzt mitglied auf true, die Bedingung ist immer wahr – auch Nichtmitglieder zahlen die Hälfte: preis(30, false) liefert 4 statt 8. Korrektur: if (mitglied) oder if (mitglied == true).

Fehler 3: p / 2 ist ganzzahlige Division und rundet ab: preis(10, true) liefert 1 statt 2. Korrektur: (p + 1) / 2.

```
public static int preis(int alter, boolean mitglied) {
    int p;
    if (alter < 6) {
        p = 0;
    } else if (alter < 18) {
        p = 3;
    } else {
        p = 8;
    }
    if (mitglied) {
        p = (p + 1) / 2;
    }
    return p;
}
```

W0-12 Das Zahlenschloss · Tracetabelle

a) Rückgabewert: 1704 (4 Durchläufe).

b) Die Methode kehrt die Ziffernfolge einer nicht negativen Zahl um: n % 10 trennt die letzte Ziffer ab, n / 10 entfernt sie; e * 10 schiebt die bisherigen Ziffern eine Stelle nach links und macht Platz für die neue Ziffer.

c) geheim(1200) liefert 21 (e: 0 → 0 → 2 → 21). Die umgedrehte Folge wäre 0021 – führende Nullen gibt es bei Ganzzahlen nicht.

```
Durchlauf | n % 10 |   e  |   n
----------+--------+------+-----
    -     |   -    |    0 | 4071
    1     |   1    |    1 |  407
    2     |   7    |   17 |   40
    3     |   0    |  170 |    4
    4     |   4    | 1704 |    0
```

W0-13 Inventur in der Vorratskammer · Funktionsweise erläutern

a) Ausgaben: 4 · 2 · 4 · 9. Die letzte Zeile bricht mit einem Laufzeitfehler ab (ArrayIndexOutOfBoundsException): Die gültigen Indizes sind 0 bis vorrat.length − 1 = 3, vorrat[4] gibt es nicht.

b) Bei for-each ist x eine eigene Variable, in die nacheinander eine Kopie jedes Wertes geschrieben wird. x = x * 2 ändert nur diese Kopie, nicht das Fach der Reihung. Zum Verändern braucht man den Index (siehe unten).

c) Die Schleife beginnt bei i = 1 – vorrat[0] wird nie addiert. Ergebnis 9 statt 13. Korrektur: int i = 0 (oder for (int x : vorrat) summe = summe + x;).

```
for (int i = 0; i < vorrat.length; i++) {
    vorrat[i] = vorrat[i] * 2;
}
```

W0-14 Zwei Schilder, ein Regal · Objektkarten und Referenzen

b) a = [9, 5, 8] · b = [9, 5, 8] · c = [3, 7, 8] · x = 0. int[] b = a kopiert nur die Referenz: a und b sind zwei Schilder an demselben Regal, b[0] = 9 ändert also auch a[0]. c ist eine echte Kopie (neue Reihung, Werte einzeln kopiert) – c[1] = 7 wirkt nur auf c. x erhält eine Kopie des Wertes 8; x = 0 ändert a nicht.

c) a == b ist true: Beide Variablen verweisen auf dieselbe Reihung. a == c ist false, obwohl c anfangs dieselben Werte hatte – == vergleicht bei Reihungen die Referenzen, nicht die Inhalte.



W0-15 Die verbotenen Bücher · Verändern und Erweitern

a) toLowerCase gehört nicht zu den erlaubten Zeichenketten-Operationen. k.substring(i, i + 1) == "a" vergleicht zwei Zeichenketten mit == – in Java werden dabei Referenzen verglichen; substring liefert ein neues Objekt, der Vergleich ist immer false und die Methode liefert 0. (Die Online-IDE vergleicht mit == inhaltlich – im Abitur gilt trotzdem: Zeichenketten immer mit equals!) substring selbst ist erlaubt.

c) Erlaubt: length, charAt, contains, compareTo, (int) Zeichen (ASCII-Wert). Nicht erlaubt: split, indexOf, trim.

```
public static int anzahlA(String s) {
    int n = 0;
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (c == 'a' || c == 'A') {
            n++;
        }
    }
    return n;
}
```

W0-16 Zauberformeln übersetzen · Methodenkopf und Signatur

a) siehe Code-Block (1) bis (4) – „nichts“ wird zu void, Reihung von Ganzzahl zu int[].

b) (5) kopiere(quelle: Reihung von Ganzzahl, anzahl: Ganzzahl): Reihung von Ganzzahl

(6) mitAnfang(titel: Reihung von Zeichenkette, c: Zeichen): Reihung von Zeichenkette

(7) gleich(x: Kommazahl, y: Kommazahl): Wahrheitswert

c) Rückgabetyp int[] (Reihung von Ganzzahl) · Methodenname kopiere · Parameter quelle vom Typ int[] und anzahl vom Typ int. public static: öffentlich sichtbar, Klassenmethode (Aufruf ohne Objekt).

```
(1) public static boolean istVoll(int[] r)
(2) public static double mittelwert(int a, int b)
(3) public static void zeigeTitel(String titel, int band)
(4) public static char erstesZeichen(String s)
```

W0-17 Passt der Schlüssel ins Schloss? · Methodenkopf und Signatur

a) Akzeptiert: (A), (E), (G), (H). Abgelehnt: (B) Der Rückgabewert ist eine Kommazahl; sie passt nicht ohne Cast in eine Ganzzahl-Variable (möglicher Verlust der Nachkommastellen). (C) Reihenfolge vertauscht: true passt nicht zum Parameter preis (Ganzzahl), 20 nicht zu mitglied (Wahrheitswert). (D) 19.5 ist keine Ganzzahl. (F) Es fehlt ein Argument – Anzahl, Reihenfolge und Typen müssen zur Parameterliste passen.

(E): Die Kommazahl wird an die Zeichenkette angehängt. (G): 3 > 2 ist ein Wahrheitswert; das Ergebnis von rabatt wird mit 2 verglichen – das Ergebnis ist ein Wahrheitswert.

b) Parameter (formale Parameter) sind die Variablen im Methodenkopf: preis und mitglied. Argumente sind die konkreten Werte beim Aufruf: 20 und true. Beim Aufruf erhält preis den Wert 20 und mitglied den Wert true.

c) Der Rückgabewert wird berechnet, aber weder gespeichert noch ausgegeben – er geht verloren.

W0-18 Original oder Kopie? · Funktionsweise erläutern

a) Ausgabe: 5 2 6

b) verdopple: Wertübergabe – x ist eine Kopie von a; die Änderung von x wirkt nicht nach außen, a bleibt 5.

verdoppleAlle: r erhält eine Kopie der Referenz und verweist auf dieselbe Reihung wie t. Die Änderung des Inhalts r[i] wirkt nach außen: t = [2, 4, 6].

ersetze: r wird auf eine neue Reihung umgelenkt – das ändert nur die lokale Kopie der Referenz. t verweist weiterhin auf [2, 4, 6].

c) Ergebnis zurückgeben statt im Parameter ändern:

```
public static int verdopple(int x) {
    return x * 2;
}
// Aufruf im Hauptprogramm:
a = verdopple(a);
```

W0-19 Würfelpunkte der Akademie · Tracetabelle

a) Rückgabewert von auswerten: 7.

b) return beendet die Methode sofort. Bei wurf = 6 ist die erste Bedingung wahr, return 3 wird ausgeführt – die zweite Abfrage wird gar nicht mehr erreicht.

c) z. B.: Die Regel steht an genau einer Stelle (Änderung nur dort nötig) · punkte lässt sich einzeln testen · auswerten bleibt kurz und lesbar · punkte kann von anderen Methoden wiederverwendet werden.

```
i | Aufruf    | Rückgabe | summe
--+-----------+----------+------
- |     -     |    -     |   0
0 | punkte(4) |    1     |   1
1 | punkte(6) |    3     |   4
2 | punkte(3) |    0     |   4
3 | punkte(6) |    3     |   7
4 | punkte(5) |    0     |   7
```

W0-20 Nur die geraden Runen · Code ↔ Struktogramm

a) return n % 2 == 0; – Achtung: n % 2 == 1 als Test auf „ungerade“ ist bei negativen Zahlen falsch (−7 % 2 ergibt −1).

c) Ergebnis [8, −2, 6]. Die Länge einer Reihung muss beim Erzeugen feststehen. Deshalb zählt die erste Schleife die geraden Werte, erst dann kann erg mit passender Länge erzeugt werden. k ist der Schreibindex in erg – er ist nicht derselbe wie die Position in r.

```
public static boolean istGerade(int n) {
    return n % 2 == 0;
}

public static int[] nurGerade(int[] r) {
    int anzahl = 0;
    for (int x : r) {
        if (istGerade(x)) {
            anzahl = anzahl + 1;
        }
    }
    int[] erg = new int[anzahl];
    int k = 0;
    for (int x : r) {
        if (istGerade(x)) {
            erg[k] = x;
            k = k + 1;
        }
    }
    return erg;
}
```

W0-21 Der Compiler streikt · Fehler im Java-Code

a) indexVon: Ist r leer, wird der Schleifenrumpf nie ausgeführt – nach der Schleife folgt kein return. bewertung: Der Compiler prüft nicht, ob die beiden Bedingungen zusammen alle Fälle abdecken. Für ihn gibt es den Weg „beide Bedingungen falsch“, und dort fehlt ein return. Regel: Jeder mögliche Weg durch die Methode muss mit return enden.

b) indexVon(new int[] {3, 5}, 5) liefert −1 statt 1: Schon im ersten Durchlauf wird r[0] = 3 geprüft; weil 3 ≠ 5, gibt der else-Zweig sofort −1 zurück. Die Methode prüft immer nur das erste Element. return -1 darf erst nach der Schleife stehen.

c) Bei bewertung genügt ein einfaches else (die zweite Bedingung ist überflüssig).

```
public static int indexVon(int[] r, int x) {
    for (int i = 0; i < r.length; i++) {
        if (r[i] == x) {
            return i;
        }
    }
    return -1;
}

public static String bewertung(int punkte) {
    if (punkte >= 50) {
        return "bestanden";
    } else {
        return "nicht bestanden";
    }
}
```

W0-22 Das Rezept der Akademie · Methode implementieren

a) Kopf: public static double anteilUeber(int[] r, int grenze). Randfälle: leere Reihung (sonst Division durch 0) · Werte gleich grenze zählen nicht mit (echter Vergleich >) · kein bzw. jeder Wert über grenze → 0.0 bzw. 100.0.

Kernpunkte: Leer-Abfrage vor der Division · Zähler mit 0 starten · Schleife über alle Indizes 0 bis r.length − 1 · Rechnung mit Kommazahl (100.0) · return in jedem Pfad.

c) [12, 0, 7, 25], grenze 7: 2 von 4 Werten (12 und 25) → 50.0. 100 * anzahl / r.length mit [5, 9, 1], grenze 4: 100 * 2 / 3 = 200 / 3 = 66 – ganzzahlige Division! Als Kommazahl gespeichert ergibt das 66.0 statt 66.66… Erst 100.0 * anzahl / r.length rechnet mit Kommazahlen.

```
public static double anteilUeber(int[] r, int grenze) {
    if (r.length == 0) {
        return 0.0;
    }
    int anzahl = 0;
    for (int i = 0; i < r.length; i++) {
        if (r[i] > grenze) {
            anzahl++;
        }
    }
    return 100.0 * anzahl / r.length;
}
```